AULA 1 - LEVANTAMENTO DE REQUISITOS

---

1. REQUISITOS: O QUE SÃO?

Definição:
Requisitos são as necessidades, funcionalidades e regras que um sistema de software deve atender para ser considerado completo e funcional. Eles servem como um guia para a equipe de desenvolvimento saber exatamente o que precisa ser construído.

Por que são importantes?

Sem entender os requisitos, o sistema pode:

- Não atender às reais necessidades dos usuários
- Ter funcionalidades desorganizadas
- Apresentar falhas constantes

Exemplo Prático:

Imagine um aplicativo de delivery como o iFood. Alguns requisitos básicos seriam:

1. O usuário deve conseguir criar uma conta
2. O sistema deve mostrar restaurantes próximos à localização do usuário
3. Deve ser possível adicionar itens ao carrinho
4. O aplicativo precisa oferecer opções de pagamento

1.1. Definição Detalhada

Requisitos precisam ser:

- **Claros:** Qualquer pessoa da equipe deve entender
- **Precisos:** Sem ambiguidades ou interpretações erradas
- **Testáveis:** Deve ser possível verificar se foram implementados

Classificação Básica:

- **Obrigatórios:** Essenciais para o sistema funcionar
  Ex.: Um site de vendas precisa ter carrinho de compras
- **Desejáveis:** Melhorias que podem ser adicionadas depois
  Ex.: O mesmo site poderia ter um chat para atendimento

Graus de Impacto:

Classifica a importância/criticidade de cada requisito no projeto.

Níveis Comuns:

1. **Crítico**
    - Falha bloqueia o sistema (ex.: pagamento não processa).
        - Ação: Prioridade máxima (correção imediata).
2. **Alto**
    - Impacta funcionalidades essenciais (ex.: login falha)
        - Ação: Resolver antes das entregas principais.

3. **Médio**
    - Afeta usabilidade, mas não paralisa o sistema (ex.: filtro de busca lento).
        - Ação: Planejar para próximas sprints.

4. **Baixo**
    - Melhorias cosméticas (ex.: cor de um botão).
        - Ação: Baixa prioridade (pós-lançamento).

1.2. Por que são Cruciais?

Problemas comuns sem requisitos bem definidos:

- **Retrabalho:** 40% dos erros em software ocorrem na fase de requisitos (dados do IEEE)

- **Custo:** Corrigir erro após implementação custa 100x mais do que na fase de requisitos

- **Insatisfação:** 56% dos usuários abandonam apps com funcionalidades mal implementadas

***Exemplo Prático* - iFood:**

Boa Definição	Má Definição
"Usuário deve filtrar restaurantes por: tipo de comida (múltipla seleção), preço médio (slider) e avaliação (★4+)"	"O app deve mostrar restaurantes bons"

1.3 Classificação Detalhada

Requisitos Obrigatórios vs. Desejáveis:

Critério	Obrigatórios	Desejáveis
Impacto	Sistema não funciona sem	Melhoria de qualidade
Exemplo	Login de usuário	Login com redes sociais
Prioridade	Alta	Média/Baixa
Custo de não implementar	Projeto falha	Usuários menos satisfeitos

---

2. TIPOS DE REQUISITOS, REGRAS DE NEGÓCIO E RESTRIÇÕES

2.1 Requisitos Funcionais (O QUE o sistema faz)

Explicação Detalhada:

Descrevem as funcionalidades específicas que o sistema deve executar. Devem ser escritos como ações mensuráveis.

Exemplos:

1. **Autenticação:**
    - RF01: O sistema deve permitir login com e-mail e senha

    - RF02: Deve enviar link de recuperação para e-mails cadastrados

2. **Processos Comerciais:**
    - RF10: O sistema deve calcular frete baseado em:
        - CEP de destino
        - Peso total dos produtos
        - Prazo selecionado

2.2 Requisitos Não-Funcionais (COMO o sistema funciona)

Explicação Detalhada:

Definem a qualidade do sistema. São frequentemente negligenciados, mas críticos para o sucesso.

Categorias Expandidas:

1. **Desempenho:**
    - RNF01: O sistema deve responder em ≤1.5 segundos para 95% das requisições
2. **Segurança:**
    - RNF05: Todos os dados pessoais devem ser criptografados de acordo com a LGPD
3. **Usabilidade:**
    - RNF10: O sistema deve seguir diretrizes WCAG 2.1 AA para acessibilidade

---

2.3. Regras de Negócio

São as políticas e normas específicas da empresa ou domínio do sistema.

Características:

- Definidas pelo cliente/empresa
- Podem mudar conforme as necessidades do negócio
- São diferentes dos requisitos técnicos

Exemplos:

1. "Clientes com mais de 3 compras no mês ganham 10% de desconto"
2. "Vendas acima de R$500 têm frete grátis"
3. "Usuários precisam ter 18+ para comprar produtos restritos"

Diferença para Requisitos Funcionais:

Requisito Funcional	Regra de Negócio
"Sistema deve aplicar desconto"	"Desconto de 15% para compras acima de R$200"

2.4. Restrições

São limitações que afetam como o sistema será desenvolvido.

Tipos Comuns:

1. Tecnológicas:

- "O sistema deve usar banco de dados MySQL"
- "Aplicativo precisa funcionar em Android 10 ou superior"

2. De Tempo:

- "Primeira versão precisa estar pronta em 3 meses"

3. Orçamentárias:

- "Não podemos gastar mais que R$20.000 com hospedagem"

Exemplo Prático:

Para um **sistema escolar:**

- "Deve funcionar em computadores com Windows 10" (restrição tecnológica)
- "Precisa ser acessível para pessoas com daltonismo" (restrição de acessibilidade)

Resumo até aqui:

✅ **Requisitos =** O que o sistema precisa fazer.

✅ **Funcionais =** Ações do sistema (ex.: cadastrar, calcular).

✅ **Não Funcionais =** Qualidade (rápido, seguro, fácil de usar).

✅ **Regras de Negócio =** Normas específicas da empresa ou projeto

✅ **Restrições =** Limitações específicas da empresa ou projeto.

---

3. MODELOS DE DOCUMENTAÇÃO AVANÇADOS

3.1 Diagrama de Casos de Uso: Guia Completo

O que é um Diagrama de Casos de Uso?

É uma representação visual que mostra **quem** interage com o sistema (atores) e **o que** esses atores podem fazer (casos de uso). É como um mapa das principais funcionalidades do sistema do ponto de vista do usuário.

Por que usar?

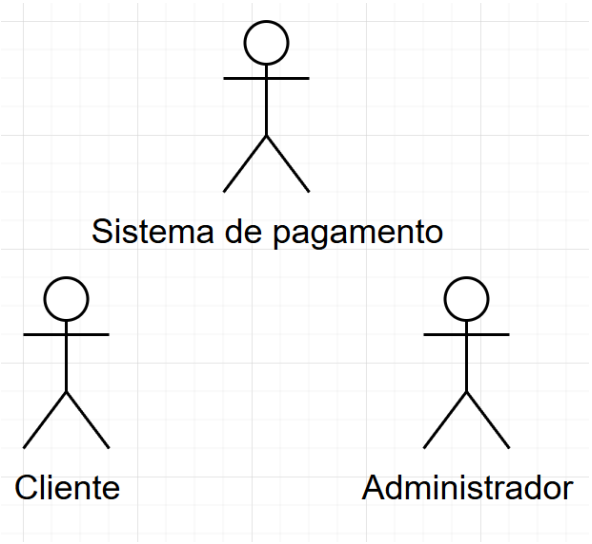
- Identifica todas as funcionalidades principais
- Mostra claramente quem usa cada parte do sistema
- Ajuda a descobrir requisitos ocultos

Elementos Fundamentais (com símbolos e explicação)

1. Atores (Quem usa o sistema)

- **Símbolo:** 👤 ou retângulo com "ator" escrito dentro
- **O que são:** Qualquer pessoa, sistema externo ou dispositivo que interage com seu software
- **Tipos:**
  - **Primário:** Usuário direto (ex.: cliente em um e-commerce)
  - **Secundário:** Apoio (ex.: sistema de pagamento)
  - **Externo:** Outro software (ex.: API de correios)

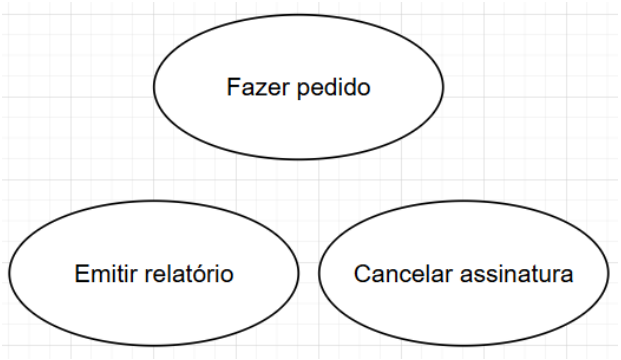
Exemplo:



2. Casos de Uso (O que o sistema faz)

- **Símbolo:** Elipse ( ) ou retângulo com borda oval
- **Regra de ouro:** Sempre começa com verbo no infinitivo
- **Níveis:**
    - **Resumo:** "Gerenciar pedidos" (alto nível)
    - **User Goal:** "Realizar pagamento" (nível médio)
    - **Subfunção:** "Validar cartão" (detalhado)

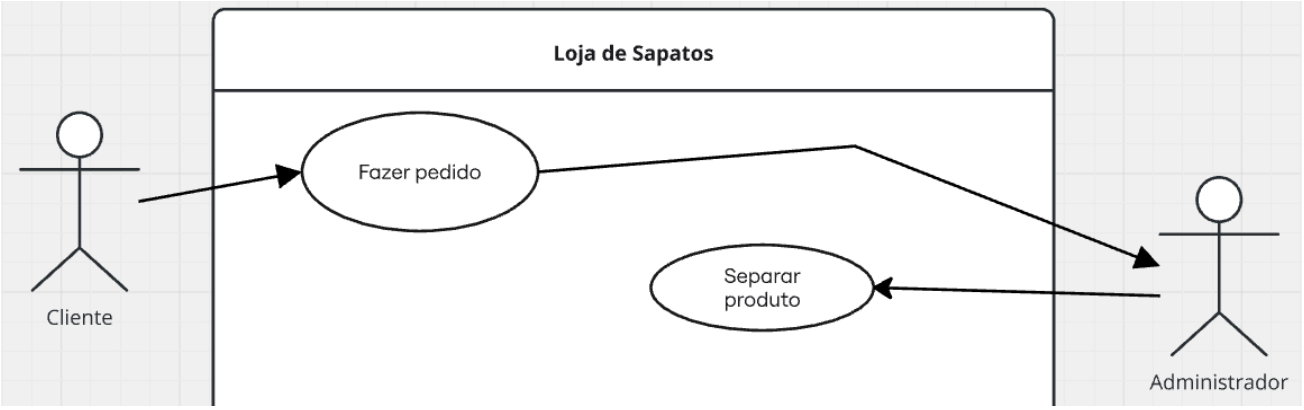
Exemplo:



3. Sistema (Limites do que está sendo modelado)

- **Símbolo:** Retângulo grande envolvendo os casos de uso
- **Título:** Nome do sistema no topo
- **Importante:** Tudo dentro está no sistema, fora são atores

Visualização:

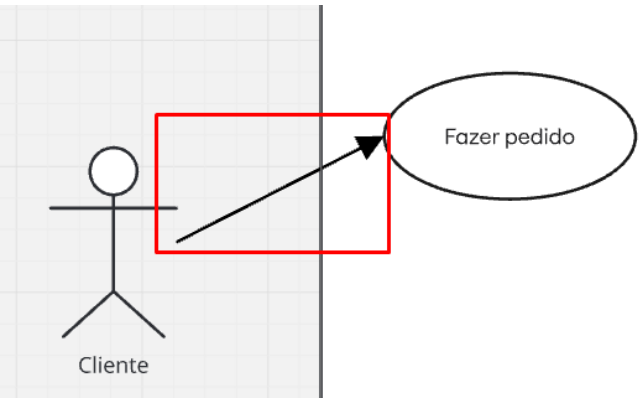


4. Relacionamentos (Como elementos se conectam)

a) Associação (Ator ↔ Caso de Uso)

- Linha simples: Mostra qual ator executa qual caso de uso

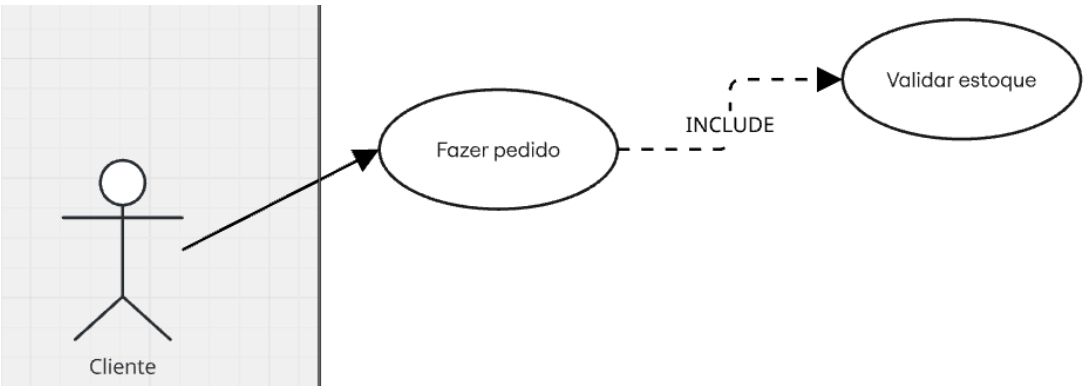
Exemplo:



b) Inclusão (<<include>>)

- Quando um caso de uso **precisa** de outro para funcionar

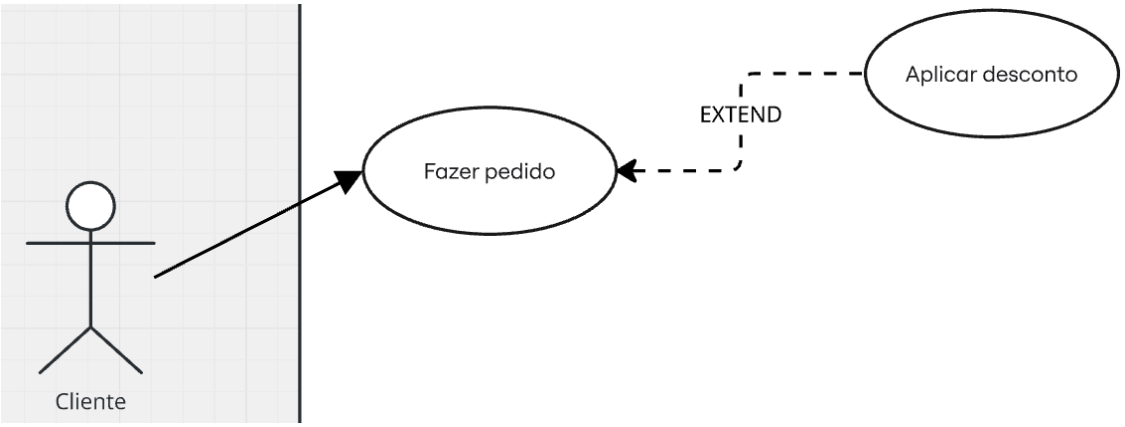
Exemplo:



c) Extensão (<<extend>>)

- Funcionalidade **opcional** que pode acontecer

Exemplo:



5. Posicionamento dos atores em casos de uso

a) Atores à ESQUERDA

- **Quem são:** Usuários ativos (humanos ou sistemas que iniciam a ação).

Exemplos: Cliente, Administrador

- **Regra:** Sempre que o ator dispara o caso de uso principal.

b) Atores à DIREITA

- Quem são: Sistemas passivos (respondem a ações, mas não iniciam).

Exemplos: Banco de Dados, API de Pagamento, Impressora.

- **Regra:** Quando o ator é acionado pelo sistema ou usuário.

c) Fluxo Básico (Esquerda → Direita)

[Ator Ativo] → (Caso de Uso) → [Ator Passivo]

d) Exceções:

- Se houver muitos atores, posicione **os mais relevantes** à esquerda.
- Atores secundários (ex.: "Serviço de Logs") podem ficar abaixo do caso de uso.

Resumo Final:

Lado	Tipo de Ator	Seta no Diagrama
Esquerda	Ativo (inicia ação)	Ator → Caso de Uso
Direita	Passivo (reage)	Caso de Uso → Ator

6. Generalização

Em diagramas de caso de uso, o termo **Generalização** (também chamado de **Herança**) representa um relacionamento entre um caso de uso mais genérico (pai) e um caso de uso mais específico (filho), onde o caso de uso filho herda e estende o comportamento do caso de uso pai.

Características da Generalização em Casos de Uso:

1. **Herança de Comportamento:**

    - O caso de uso filho herda as funcionalidades do caso de uso pai.
    - O filho pode adicionar novos comportamentos ou modificar os existentes.

2. **Notação:**

    - Representado por uma seta com linha contínua e ponta aberta (→) apontando do caso de uso filho para o pai.

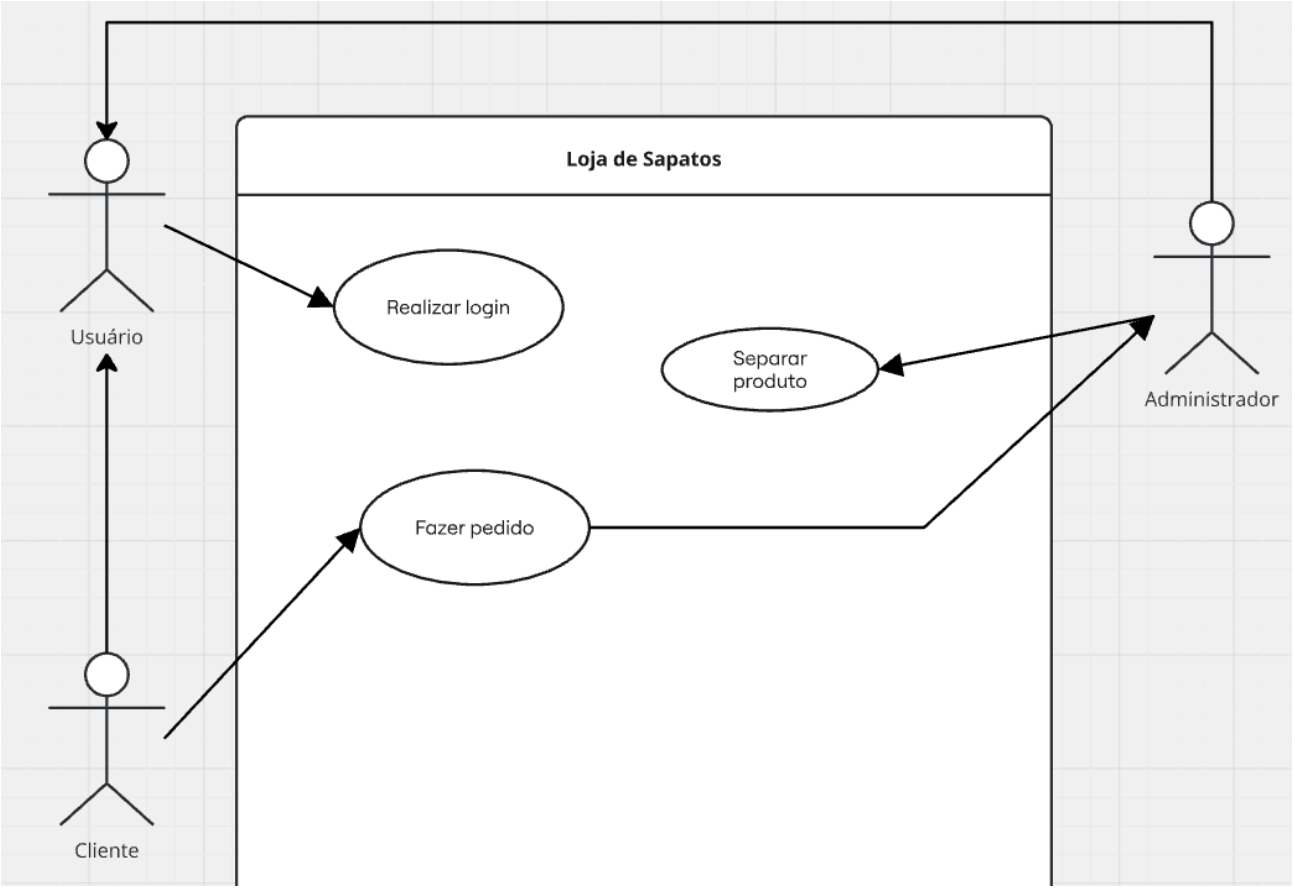
3. **Propósito:**

    - Evitar repetição de funcionalidades comuns.
    - Organizar casos de uso de forma hierárquica, facilitando a compreensão.

Exemplo: **Sistema de Login com Generalização**

Contexto:

- **"Usuário"** é um caso de uso genérico (pai), que representa ações básicas como **"Fazer Login".**
- **"Administrador"** e **"Cliente"** são casos de uso específicos (filhos), que herdam **"Fazer Login"** mas têm funcionalidades adicionais exclusivas.



Por que isso é útil?

- **Evita repetição:** Não precisamos descrever "Fazer Login" duas vezes (para Cliente e Administrador).

- **Organiza o sistema:** Mostra claramente que Administrador e Cliente são tipos de Usuário.

- **Facilita manutenção:** Se a regra de login mudar, basta alterar no Usuário (pai).

7. Passo a Passo para Criar (com exemplo real)

Sistema: Aplicativo de Banco

1. **Listar atores:**

    - Cliente
    - Caixa eletrônico
    - Gerente

2. **Identificar casos de uso principais:**

    - Realizar saque
    - Solicitar empréstimo
    - Consultar saldo
    - Liberar saque
    - Atualizar software
    - Aprovar empréstimo

Se o caixa eletrônico só processa o saque sem retorno explícito, vincule-o diretamente a "Realizar Saque".

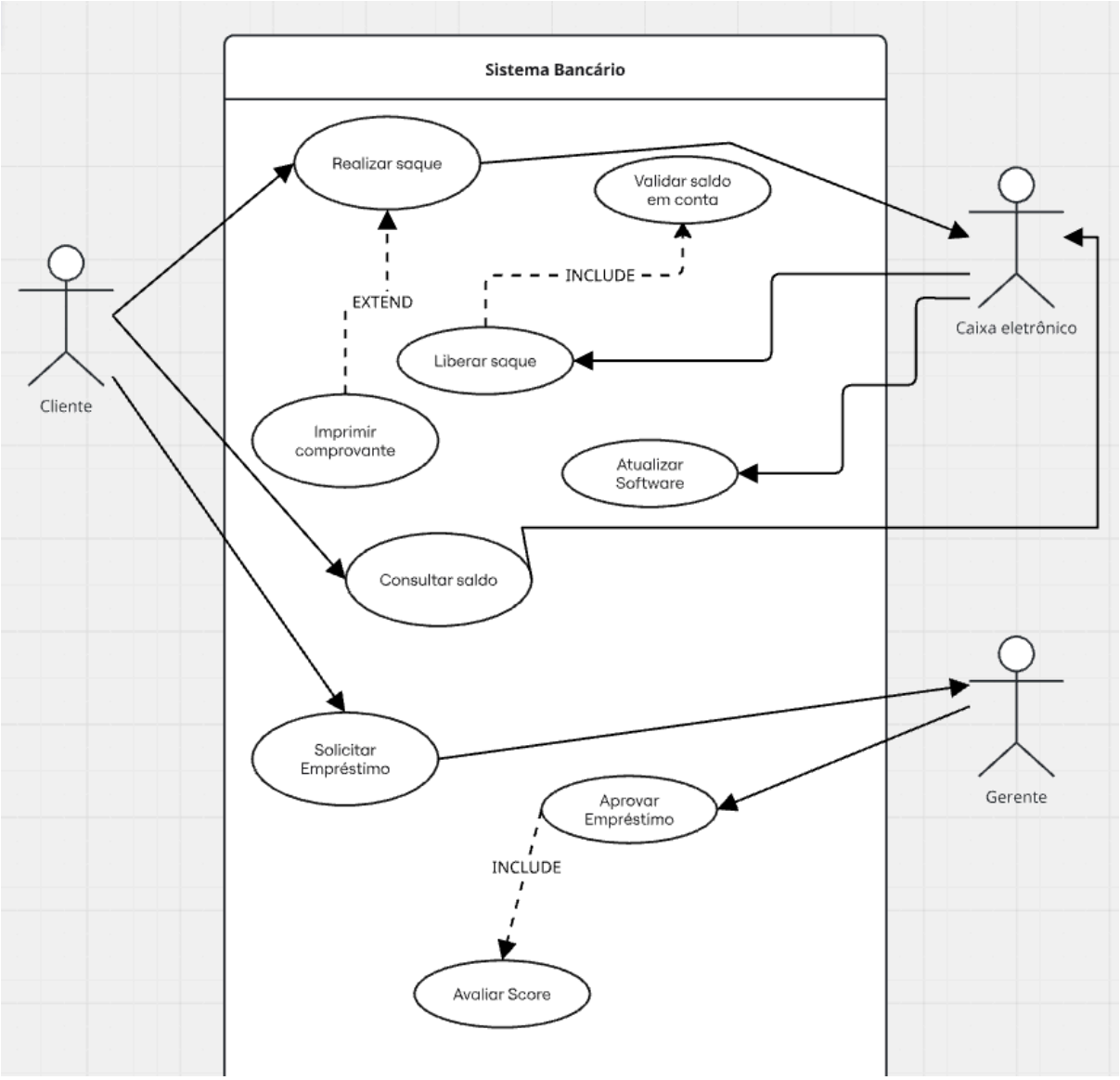
Se ele tem comportamentos ativos (como liberar dinheiro), modele como ação separada.

3. **Adicionar relacionamentos complexos:**

(Realizar saque) <<extend>>(imprimir comprovante)
(Liberar saque) <<include>> (Validar saldo em conta)
(consultar saldo) <<extend>> (Notificar destinatário)
(aprovar empréstimo)<<include>>(Avaliar score)

4. **Produzir o diagrama:**

Após toda a análise das necessidades (requisitos) adicionar os atores, casos de uso e includes e extends.



Erros Comuns (e como evitar)

1. **Muitos casos de uso no mesmo diagrama**
   Solução: Divida em sub diagramas por módulo

2. **Atores que não interagem**
   Solução: Remova ou mostre relação real

3. **Casos de uso muito vagos**
   Antes: "Gerenciar usuários"
   Depois: "Cadastrar usuário", "Resetar senha"

3.2 Matriz de Rastreabilidade: Guia Prático

O que é, e por que é vital?

Tabela que liga cada requisito a:

- Sua origem (quem pediu)
- Onde foi implementado (caso de uso)
- Como foi testado
- Status atual

Benefícios:

- 100% de cobertura - nada fica esquecido
- Impacto de mudanças visível imediatamente
- Prova de que todos requisitos foram atendidos

ID	Descrição	Tipo	Grau de impacto	Fonte	Caso de Uso	Teste	Sprint	Status
RF01	Login com e-mail	Funcional	Alto	Reunião 15/03	UC-01	CT-10	1	✔️
RNF05	Tempo resposta <2s	Não-func	Médio	Documento ARQ	-	PT-05	3	⚠️

Estrutura Detalhada (coluna por coluna)

Explicação de cada campo:

1. **ID:** Código único (RF = Requisito Funcional, RNF = Não-funcional)
   Padrão recomendado: RF001, RNF010
2. **Descrição:**
   - Para funcionais: "O sistema deve [ação]"
   - Para não-funcionais: Métrica clara ("Disponibilidade 99.9%")
3. **Tipo:**
   - Funcional
   - Não-funcional (performance, segurança, etc.)
4. **Grau de impacto:**
   - Crítico
   - Alto
   - Médio
   - Baixo
5. **Fonte:**
   - Reunião com [cliente] em [data]
   - Documento [nome]
   - Regulamento [ex.: LGPD]

6. **Caso de Uso (UC):**
    - Link para o diagrama (ex.: UC-01 = "Fazer login")
    - Não-funcionais podem ficar em branco
7. **Teste:**
    - CT = Caso de Teste (funcional)
    - PT = Plano de Teste (não-funcional)
8. **Sprint:**
    - Quando foi/será implementado
    - Útil em metodologias ágeis
9. **Status:**
    - ✔️ Implementado
    - ⚠️ Pendente
    - ❌ Cancelado

Exemplo Prático - Sistema de Hotel

ID	Descrição	Tipo	Grau de impacto	Fonte	Caso de Uso	Teste	Sprint	Status
RF10	Reservar quarto	Func	Alto	Cliente	UC-05	CT-20	2	✔️
RF11	Cancelar reserva	Func	Alto	Cliente	UC-06	CT-21	2	⚠️
RNF15	Check-in em <5min	Não-func	Médio	Benchmark	-	PT-11	4	❌

Como preencher na prática:

1. Liste **TODOS** os requisitos do seu documento
2. Para cada um, identifique:
    - De onde veio (fonte)
    - Grau de impacto
    - Qual caso de uso resolve (se funcional)
3. Durante o desenvolvimento:
    - Atualize a sprint
    - Marque os testes realizados
4. Na entrega:
    - Todos devem estar ✔️ ou justificados

3.3 Documentação ABNT: Vantagens Práticas

1. Padronização Nacional

- **Estrutura uniforme:** Todos os documentos técnicos seguem o mesmo formato (títulos, margens, numeração)
- **Facilidade de leitura:** Informações organizadas de forma lógica e previsível

  Exemplo:

  - Capa com elementos obrigatórios (título, autor, instituição)
  - Sumário com hierarquia clara (1.1, 1.1.1)

2. Aceitação em Instituições

- **Obrigatório em:** TCCs, artigos científicos, projetos de pesquisa
- **Vantagens:**
  - Atende exigências de universidades
  - Prepara para padrões profissionais
  - Evita reprovação por formatação

3. Organização Profissional

- **Aplicações:**
  - Propostas comerciais
  - Manuais técnicos
  - Documentação de sistemas
- **Benefícios:**
  - Transmite profissionalismo
  - Facilita colaboração em equipe
  - Agiliza revisões e aprovações

Estrutura básica ABNT para projetos:

1. Capa
2. Sumário
3. Introdução
4. Objetivos
5. Especificação de requisitos
6. Modelos conceituais
7. Referências

Exemplo prático de formatação:

3. REQUISITOS FUNCIONAIS

3.1 RF01 - Cadastro de usuários

Descrição: O sistema deve permitir o cadastro de novos usuários com os seguintes campos obrigatórios:

- Nome completo (mín. 3 caracteres, máx. 100)
- E-mail válido (com validação de formato)
- Senha (mín. 8 caracteres, com 1 número e 1 caractere especial)

Prioridade: Alta (P1)

Origem: Reunião com cliente em 15/03/2024 (ATA-023, página 7)

Regra de Negócio: RN005 - Usuários menores de 16 anos precisam de responsável legal

Caso de Uso Relacionado: UC-02 (Cadastro básico)

Teste: CT-012 (Validação de formulário)

Observações: Campos opcionais: telefone e data de nascimento

4. REQUISITOS NÃO-FUNCIONAIS

4.1 RNF01 - Desempenho

Descrição: Todas as páginas devem carregar em menos de 2 segundos para conexões de 5Mbps, com até 500 usuários simultâneos

Categoria: Performance

Métrica: Tempo médio de resposta (2 segundos)

Prioridade: Crítica (P0)

Origem: Benchmark do setor (Padrão Web Consortium, 2023)

Teste: PT-101 (Teste de carga com JMeter)

Observações: Exceto durante manutenções programadas

Explicação dos Campos nos Requisitos

Campos Comuns a Ambos (RF e RNF)

1. **Descrição**
    - O que o sistema deve fazer/garantir
    - Detalhes técnicos ou regras específicas
2. **Prioridade**
    - P0 (Crítica): Imprescindível para o sistema funcionar
    - P1 (Alta): Essencial, mas pode ter workaround temporário
    - P2 (Média): Importante, mas pode ser postergado
    - P3 (Baixa): Melhoria ou "bom ter"
3. **Origem**
    - De onde surgiu o requisito (ex: cliente, lei, equipe técnica)
4. **Teste**
    - CT-XXX: Caso de Teste (para requisitos funcionais)
    - PT-XXX: Plano de Teste (para não-funcionais)
5. **Observações**
    - Informações complementares ou exceções

Campos Exclusivos dos Requisitos Funcionais

6. Regra de Negócio (RN)
    - Norma ou política que justifica o requisito
    - Ex: RN005: Validação de idade mínima
7. Caso de Uso Relacionado (UC)
    - Link para o diagrama/fluxo correspondente
    - Ex: UC-02: Fluxo de cadastro de usuário

Campos Exclusivos dos Requisitos Não-Funcionais

8. **Categoria**
    - Classificação do tipo de exigência:
        - Performance (tempo de resposta, capacidade)
        - Segurança (criptografia, autenticação)
        - Usabilidade (acessibilidade, intuitividade)
        - Compatibilidade (navegadores, dispositivos)
9. **Métrica**
    - Como o requisito será medido/testado:
        - Ex: Tempo de carregamento < 2 segundos
        - Ex: Disponibilidade de 99.9% (uptime)

3.4 Protótipos de Interface

Os protótipos de interface são representações visuais das telas de um sistema, usados para validar ideias, testar a usabilidade e comunicar o design antes do desenvolvimento.

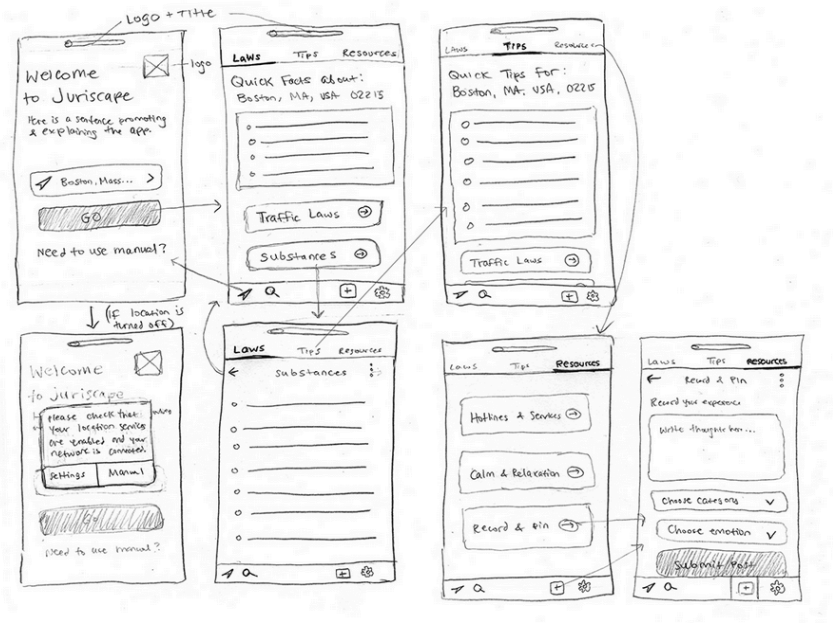
Tipos de Protótipos (Comparação por Fidelidade)

Tipo	Fidelidade	Características	Quando Usar	Ferramentas Ex.
Wireframe	**Baixa**	- Esboço simples (preto e branco) - Foco em **layout e estrutura** - Sem cores ou detalhes visuais	Brainstorming inicial Validação rápida do fluxo	Papel, Balsamiq, Figma (em wireframe)
Mockup	**Média**	- Versão mais detalhada - Cores, tipografia e elementos visuais - **Parece real, mas não é funcional**	Apresentação para stakeholders Refinamento do visual	Figma, Adobe XD, Sketch
Protótipo Clicável	**Alta**	- Simula **interações reais** (botões, navegação) - Pode incluir animações e microinterações	Testes de usabilidade Validação final antes do desenvolvimento	Figma (Prototype), Proto.io, Framer

Comparação Detalhada

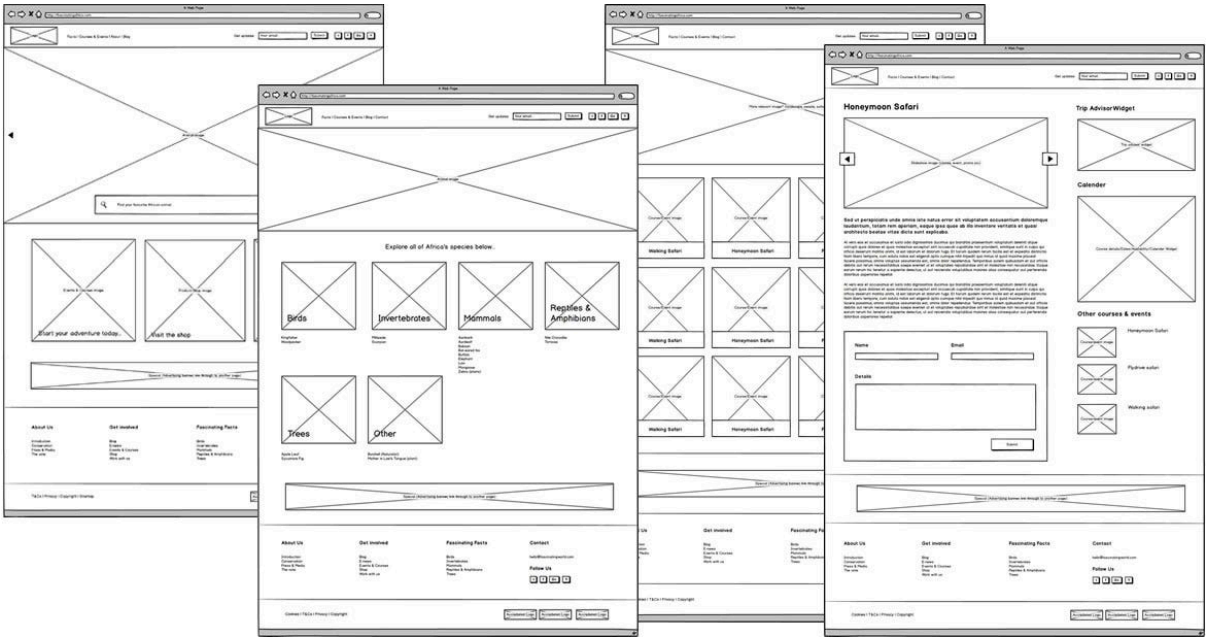
1. **Wireframe (Baixa Fidelidade)**
   - **Objetivo:** Definir estrutura básica (onde os elementos ficam).
   - **Prós:** Rápido, barato, fácil de modificar.
   - **Contras:** Não mostra experiência real.

Exemplo:



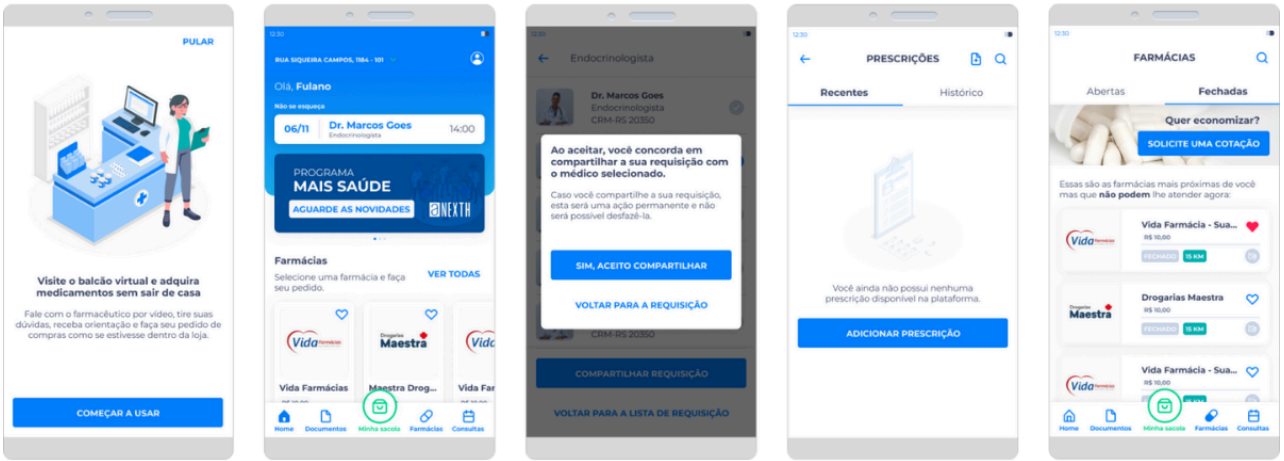
2. **Mockup (Média Fidelidade)**
    - **Objetivo:** Refinar aparência visual (cores, tipografia, imagens).
    - **Prós:** Mais próximo do resultado final, bom para aprovação.
    - **Contras:** Ainda não é interativo.

Exemplo:



3. **Protótipo Clicável (Alta Fidelidade)**
    - **Objetivo:** Testar funcionalidade e usabilidade.
    - **Prós:** Simula a experiência real, identifica problemas antes do código.
    - **Contras:** Demora mais para criar.

Exemplo:



4. CONTROLE DE VERSÃO PARA DOCUMENTAÇÃO DE REQUISITOS (NÃO-TÉCNICO)

4.1 Sistema de Versionamento Manual para Documentos

Objetivo Principal:

Manter um histórico organizado das mudanças nos requisitos, permitindo:

- Rastrear quando cada mudança foi feita
- Voltar a versões anteriores se necessário
- Evitar trabalho em documentos desatualizados

4.2 Padronização de Nomes de Arquivo

Estrutura Básica:

[Nome do Projeto]_[Tipo Doc]_v[Versão].[Subversão]_[Status]_[Data].ext

***Exemplo Prático*:**

Ecommerce_Requisitos_v2.3_Aprovado_15052024.docx

Cada Parte Explicada:

1. **Nome do Projeto (Ex: Ecommerce, SistemaEscolar)**

    - Máximo 3 palavras
    - Sem espaços ou caracteres especiais

2. **Tipo de Documento (Ex: Requisitos, CasosDeUso)**

    - **Tipos comuns:**
        - ReqFuncionais (Requisitos Funcionais)
        - ReqNaoFunc (Requisitos Não-Funcionais)
        - DiagramasUC (Diagramas de Casos de Uso)

3. **Versão (vX.Y):**

    - **X (Versão Principal):**
        - Incrementa em mudanças significativas
        - Nova fase do projeto
        - Mudanças estruturais no documento
    - **Y (Subversão):**
        - Pequenos ajustes
        - Correções de texto
        - Atualizações menores

4. **Status:**

    - **Rascunho:** Em edição
    - **Validado:** Revisado pela equipe
    - **Aprovado:** Aprovado pelo cliente/gestor

5. **Data (DDMMAAAA):**

    - Data da última modificação
    - Formato fixo sem barras

***Exemplos Práticos*:**

1. SistemaRH_ReqFuncionais_v1.0_Rascunho_20042024.docx
2. AppDelivery_DiagramasUC_v3.2_Validado_05052024.pdf

4.3 Quando Criar Nova Versão

Atualizar Subversão (v1.0 → v1.1):

- Correção de erros de digitação
- Ajustes de formatação
- Pequenas clarificações no texto

Atualizar Versão Principal (v1.3 → v2.0):

1. Mudanças nos Requisitos:
    - Adição/remoção de funcionalidades
    - Alteração em regras de negócio importantes
2. Aprovações Formais:
    - Quando o cliente aprova uma versão
    - Após revisões importantes
3. Marcos do Projeto:
    - Finalização da fase de levantamento
    - Início do desenvolvimento

4.4 Estrutura de Pastas Recomendada

```
📁 Documentacao_ProjetoX
├── 📁 Versoes
│   ├── 📄 ProjetoX_Requisitos_v1.0_Rascunho.docx
│   ├── 📄 ProjetoX_Requisitos_v1.1_Validado.pdf
│   └── 📄 ProjetoX_Requisitos_v2.0_Aprovado.docx
├── 📁 Anexos
│   ├── 📁 Diagramas
│   └── 📁 Referencias
└── 📄 LEIA-ME.txt (explica estrutura)
```

4.5 Controle de Alterações por Versão

Registro em Planilha (Controle Simplificado):

Versão	Data	Alterações Principais	Autor	Páginas Afetadas
v1.0	10/04/2024	Documento inicial	João	Todas
v1.1	15/04/2024	Adicionados RF50-RF55	Maria	7-9
v2.0	01/05/2024	Revisão completa com cliente	Carlos	Todas

Registro Detalhado (Dentro do Documento):

=== HISTÓRICO DE VERSÕES ===

v2.0 - 01/05/2024 - APROVADO

• Adicionada seção de requisitos legais (páginas 12-15)

• Removido requisito RF33 por decisão do cliente

• Diagramas atualizados conforme reunião 28/04

v1.1 - 15/04/2024 - VALIDADO

• Correção de erros nos casos de uso UC10-UC12

• Formatação padronizada em todas as tabelas

v1.0 - 10/04/2024 - RASCUNHO

• Versão inicial do documento

4.6 Boas Práticas Essenciais

1. Nunca edite versões aprovadas
    - Sempre crie uma cópia com nova versão
2. Mantenha versões anteriores por 6 meses (Ou conforme política da empresa)
3. Use converter para PDF nas versões validadas
    - Evita alterações acidentais
4. Padrão de cores para status:
    - 🔴 **Rascunho (em edição)**
    - 🟡 **Validado (aguardando aprovação)**
    - 🟢 **Aprovado (versão final)**
5. Regra dos 7 dias:
    - Se um documento ficar mais de 7 dias como "Rascunho", deve ser revisado

---

4.7 Exemplo Completo de Ciclo de Vida

1. **Primeira Versão:**

ProjetoY_ReqFunc_v1.0_Rascunho_01062024.docx

2. **Após Revisão da Equipe:**

ProjetoY_ReqFunc_v1.1_Validado_08062024.pdf

3. **Cliente Solicita Mudanças:**

ProjetoY_ReqFunc_v1.2_Rascunho_10062024.docx

4. **Versão Final Aprovada:**

ProjetoY_ReqFunc_v2.0_Aprovado_15062024.pdf

---

5. RESUMO FINAL

5.1 O que são Requisitos?

- **Definição**: Necessidades, funcionalidades e regras que o sistema deve atender.
- **Importância**: Evitam retrabalho, custos extras e insatisfação do usuário.
- **Classificação**:
  - **Obrigatórios**: Essenciais (ex.: carrinho de compras).
  - **Desejáveis**: Melhorias (ex.: chat de atendimento).

5.2 Tipos de Requisitos

- **Funcionais (RF)**: O que o sistema faz (ex.: "RF01: Login com e-mail").
- **Não-Funcionais (RNF)**: Como o sistema funciona (ex.: "RNF01: Tempo de resposta ≤2s").
- **Regras de Negócio**: Normas específicas (ex.: "Desconto de 10% para compras acima de R$200").
- **Restrições**: Limitações (ex.: "Só funciona em Android 10+").

5.3 Documentação

- **Diagrama de Casos de Uso**: Mostra interações entre atores e funcionalidades.
  - **Atores**: À esquerda (ativos) ou direita (passivos).
  - **Relacionamentos**: <<include>> (obrigatório) e <<extend>> (opcional).
- **Matriz de Rastreabilidade**: Liga requisitos a fontes, testes e status.
- **Protótipos**:
  - **Wireframe** (baixa fidelidade): Estrutura básica.
  - **Mockup** (média fidelidade): Visual detalhado.
  - **Protótipo clicável** (alta fidelidade): Simula interações.

5.4 Controle de Versão

- **Padrão de nomes**: ProjetoX_Requisitos_v1.0_Aprovado_15052024.docx.
- **Boas práticas**: Nunca edite versões aprovadas; mantenha histórico claro.

Objetivo Final: Garantir que o sistema atenda às necessidades reais, com clareza e organização!